

Master Technical Reference

SIH 2026 · PS 26090 · AI-Driven Market Linkage & Smart Cataloging for Marginalized Artisans

Organisation: Ministry of Social Justice and Empowerment (MoSJE) **Department:** Department of Social Justice and Empowerment **Category:** Software · **Theme:** Miscellaneous **Status:** Master reference · v1

WHAT THIS DOCUMENT DOES NOT COVER

Nothing about revenue, business model, pricing to customers, partners, or funding is in this file.

Topic	Where it lives
Revenue streams, Business Model Canvas, partners, TAM, cost displacement, pitch economics	BMC-v2-Review-Brief.md [!] <i>needs patching</i>
Condensed flow summary	Product-Flow-Architecture.md (subset of §4 & §5 here)
Everything technical, product, and PS-alignment	This file

If you are looking for "how do we make money" — **wrong file.**

READING GUIDE — who reads what

Your role	Read these sections
Everyone, day one	§1 (PS requirements), §2 (what we're building), §3 (surfaces)
Mobile / frontend dev	§3, §4 (on-device), §6 (voice UX), §14 (privacy UX)
AI / ML dev	§5 (server pipeline), §6 (NLP), §7 (pricing), §8.4 (GeM category mapping)
Backend dev	§8 (channels), §9 (orders), §10 (inventory), §11 (settlement), §15 (data model)
Whoever owns the marketplace	§3.3, §9, §10, §12
Whoever presents to judges	§1, §2, §13 (mentor answers), §14, §17 (open decisions)
Whoever talks to mentors	§13 — every mentor note is answered there
Before any slide is made	§18 (verification checklist) — nothing unverified goes on a slide

Table of Contents

- PS 26090 — verbatim requirements & scope audit
- What we are building — the Factory model
- Surfaces — app, web, marketplace
- Image capture — on-device quality gate
- Image processing — server pipeline
- Voice & multilingual cataloging
- Dynamic pricing
- Channel integration layer
- Order management — unified inbox
- Inventory sync
- Settlement visibility
- Logistics & packaging
- Mentor notes — answered
- Privacy & DPDP
- Data model
- Tech stack
- Open decisions register
- Verification checklist
- Reference numbers

1. PS 26090 — verbatim requirements & scope audit

1.1 What the PS literally demands

Deliverable shape: - "AI-powered, cross-platform mobile application" - "supported by a robust, scalable backend architecture" - "highly responsive, minimalist UI/UX design (incorporating modern, clean visual hierarchies and accessible layouts)"

Three numbered features:

#	Feature	Verbatim requirement
1	AI Image Enhancer & Studio	"built-in camera module", remove cluttered backgrounds, correct lighting, format to professional e-commerce standards. Examples given: textiles, handicrafts
2	Multilingual Auto-Cataloger	Voice notes in regional languages → translate → SEO-friendly professional descriptions in English AND Hindi
3	Dynamic Pricing Assistant	Analyses image and description → optimal competitive price based on "current market trends and raw material costs "

Requirements buried in the Challenge paragraph — these count too: - acts as a "virtual business manager" - "seamlessly **digitize their inventory**" - "optimize their listings using AI" - "connect directly with larger **B2B buyers or government e-marketplaces** without requiring advanced technical knowledge"

Impact goals: - Continuous year-round digital sales channel, reducing dependency on periodic physical fairs - Drastically lower the barrier to entry through intuitive AI automation - Improve digital literacy and financial independence, increasing average annual income

1.2 [CRITICAL] The channel misreading — fixed

The PS never mentions Amazon. Or Flipkart. Or Meesho. Or Myntra. Not once.

It says "**B2B buyers or government e-marketplaces.**"

Early planning built the entire integration strategy around consumer marketplaces. **That was backwards.**

Corrected priority:

PRIMARY	GeM (government e-marketplace) + B2B buyers
SECONDARY	ONDC (government-backed network)
BONUS	Amazon, Flipkart – where the artisan qualifies
DROPPED	Myntra, Nykaa, Ajio – brand-gated, impossible for artisans

Same code either way. **Opposite emphasis in the pitch.**

1.3 Scope audit

**** In scope, covered**** Cross-platform mobile app · AI image enhancement · voice cataloging in regional languages · SEO descriptions EN+HI · dynamic pricing · minimalist accessible UI

[!] In scope, was missing — now added

PS phrase	Was missing	Now
"digitize their inventory"	No inventory module	\$10
"government e-marketplaces"	GeM deprioritised	\$8.4 — now primary
"larger B2B buyers"	No B2B surface	\$3.3 — RFQ flow
"raw material costs"	Not in pricing plan	\$7 — cost-up model
"in English and Hindi"	Risk of only one	\$6 — both, always
"virtual business manager"	No order handling	\$9

● **Out of scope — we added it (justify or cut)**

Feature	Verdict
Own marketplace	Not asked. Keep — it's the B2B surface + zero-paperwork fallback (§3.3)
Analytics dashboard	Not asked. Keep — defensible under "virtual business manager". Cheap, demos well
Product story / provenance	Not asked. Keep small — supports impact goals, answers mentor notes
Blockchain	Not asked. One slide only (§13.8)
B2C consumer app	Not asked. Secondary to B2B

1.4 The seller-account question — settled

Literal: the PS does not say "help create seller accounts."

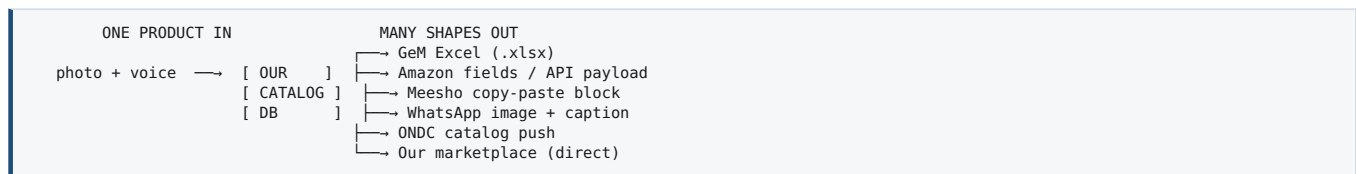
But: target users have "low digital literacy... lack of technical skills." Impact goal is to "drastically lower the barrier to entry." Someone matching that description **does not already have an Amazon Seller Central account.**

Therefore: if the app only serves artisans who already onboarded themselves, we serve people who already solved the hard problem. **Not literally required, functionally mandatory.**

Resolution: **we teach, they act.** See §4 of the flow — animation + voice video, deep-link to the platform's own site, self-reported completion. **No agents, no proxy accounts, no documents held.**

2. What we are building — the Factory model

A catalog factory. Not a shop. Not an agency.



Three rules:

1. **The catalog is ours.** Products live in our DB whether or not the artisan ever joins a platform. Edit once → all exports regenerate.
2. **We render, we don't operate.** We produce platform-shaped output. The artisan uploads it on the platform's own site.
3. **We hold no documents.** No PAN, no Aadhaar, no GST cert, no bank credentials, no platform passwords.

Every platform has a different listing schema. **Normalising that difference is the product.**

Three hard NOs, agreed and locked: - No field agents / no human visits — breaks "without requiring advanced technical knowledge" - No document storage — see §14 - No charging artisans — anything

3. Surfaces — app, web, marketplace

Three distinct surfaces. Only one is mobile.

3.1 Artisan app — mobile only

Camera-first. Offline-first. Voice-first. Zero typing.

Screens: language pick → camera → listing review → catalog list → channel unlock map → order inbox → earnings.

A web app for artisans is wasted effort. They have one shared low-end phone, patchy network, and cannot type. Everything here must work with a thumb and a voice.

3.2 Admin console — web

For cluster coordinators and agency staff. This is what our institutional users actually touch.

- Bulk artisan management
- Assisted onboarding support (remote — phone call, not visit)
- Order/GeM reconciliation (see §9)
- Impact reporting exports
- Multi-tenant: each agency sees only its own artisans

3.3 Marketplace — web

This is a B2B discovery surface and a zero-paperwork fallback. Not a competitor to Amazon.

Why it exists — technical/PS reasons only:

1. **It is the only direct B2B surface.** PS says "connect **directly** with larger B2B buyers." On Amazon we're one of a million sellers — that's indirect. A buyer browsing our artisan catalog and sending a bulk RFQ **is** the direct connection.
2. **It is the only channel that works with zero paperwork.** An artisan with no PAN cannot onboard to GeM, Amazon, or ONDC. Marketplace + shareable catalog link + UPI is the only thing that works for them. Drop it and we abandon exactly the demographic the PS targets.
3. **Bulk orders cannot be expressed anywhere else.** A shopkeeper needing 200 gamchas has no way to say that on Amazon.
4. **It is the only place the craft story can live.** Provenance, manufacture info, authenticity — Amazon's schema has no room for any of it. Kill the marketplace and every mentor requirement in §13 dies with it.
5. **It generates our own pricing data.** Real transactions feed §7.
6. **It is the only thing we can demo end-to-end.** We cannot make a real Amazon listing go live on stage. We can absolutely demo photo → catalog → live listing → buyer order → artisan voice notification.

Key flows: - Product browse / filter by craft, cluster, GI tag, material - **Bulk RFQ** — "Request bulk quote" → routed to artisan as voice notification, with cluster CC'd - Craft story pages — provenance, artisan profile, manufacture info - Digital Sahayak affiliate share links (*mechanics in business doc; here it's just a share-link + attribution surface*)

4. Image capture — on-device quality gate

4.1 The principle

Phone's job is to STOP bad photos. Server's job is to BEAUTIFY good ones.

The phone judges. The server builds. The on-device layer is **not doing AI — it is doing quality gatekeeping**.

Why this matters: if a blurry photo uploads, the artisan burns data (they pay per MB), we burn GPU time, and 20 seconds later the result is still bad — and *now* they have to re-shoot while frustrated. If the phone catches it at capture, they fix it instantly. **Every photo rejected on-device is money saved and frustration avoided.**

4.2 The frame loop

Camera delivers ~30 frames/sec to the preview. We sample **every 2nd-3rd frame** → 10-15 checks/sec, which reads as "live" to the eye.

4.3 [KEY] The downscale trick — this makes the whole feature possible

4000×3000 colour frame	(12 million pixels)
↓ shrink	
240×180 colour	(43,000 pixels – 280× less work)
↓ grayscale	
240×180 gray	(1 value/pixel instead of 3)
↓	
ONE buffer, read by ALL FOUR checks	

This works because we're measuring the photo's **condition**, not its quality. "Is it dark?" and "is it blurry?" are just as visible at 240×180.

4.4 The four checks

(1) Blur — Laplacian variance Sharp photo = brightness jumps hard between neighbouring pixels (a saree edge: black → white). Blurry photo = everything changes gradually. The Laplacian filter measures "how different is each pixel from its neighbours." Take the variance → one number. High = sharp.

[!] **Content-dependent.** A plain white cloth naturally has few edges and will score "blurry" while being sharp. So: **calibrate the threshold on our own test photos**, and treat it as a *suggestion* ("shayad dhundhla hai, dobara?"), never a hard block.

(2) Light — histogram Bucket every pixel by brightness (0-255). Then check:

Signal	Meaning
Many pixels at 250-255	Blown out — detail permanently gone
Many pixels at 0-5	Crushed — detail permanently gone
Mean < 60	Underexposed
Mean > 200	Overexposed
All bunched mid-range	Flat, no contrast

[!] **This matters more than blur.** Blur can be partly sharpened. **A blown-out white region contains no information at all** — no AI can recover it.

(3) Tilt — accelerometer, not camera No image needed. The sensor reports gravity direction as (x, y, z) → trigonometry → **roll** (horizon tilt) and **pitch** (forward/back lean).

- Flat items (dhurrie, painting, dupatta on floor) → want pitch ≈ 90° (phone looking straight down)
- Standing items (matka, murti) → want pitch ≈ 0°

[!] **Very noisy sensor.** Raw values jitter constantly even when still. **Must smooth:** `new = 0.9 × old + 0.1 × reading`. Without this the UI indicator vibrates and users give up.

This check is **completely free** — separate event stream, no camera work.

(4) Framing — grid variance (cheap method)

Don't detect the product. Detect **where something is happening**.

Split the frame into ~12×9 = 108 cells. Measure variance in each: - Empty floor/wall cell → pixels similar → **low variance** - Product cell → texture, pattern, edges → **high variance**

Draw a bounding box around the "busy" cells. `box area ÷ frame area` = fill %.

- < 40% → "paas aayein"
- > 90% → "thoda peeche"
- Off-centre → "beech me layein"

Expensive alternative: small TFLite segmentation model at 160×160, every 3rd frame. More accurate, but bigger app, slower on low-end phones, drains battery.

Decision: build the cheap one. It handles ~80% of cases, especially when the product is on a plain surface (which we're telling them to do anyway). Add the model later only if time permits.

4.5 Live feedback — where most teams fail

** Failure mode 1 — showing everything at once**

"Photo blurry hai, andhera hai, phone tedha hai, product door hai"

Four problems at once = app uninstalled.

**** Show ONE problem — the highest-priority one.**** Strict ladder:

- | | | |
|----------------|-------------------------|--|
| 1. Too dark? | → "roshni me layein" | (other checks are meaningless in the dark) |
| 2. Too bright? | → "chhaya me layein" | |
| 3. Blurry? | → "phone sthir rakhein" | |
| 4. Too far? | → "paas aayein" | |
| 5. Tilted? | → "phone seedha karein" | |
| 6. All clear | → ○ GREEN | |

Only reveal the next problem after the current one clears.

**** Failure mode 2 — UI flickering every frame**** Borderline photos oscillate red/green 10×/sec. Looks broken.

**** Hysteresis. Only change state if the new state holds ≥ 0.5 seconds**.** And use different thresholds for red→green vs green→red so it doesn't sit on the boundary.

4.6 Feedback channels — four at once

Channel	What
Colour	Ring around the screen — red / amber / green
Icon	Large tick or cross. Never rely on text
Voice	"Thoda paas aayein" — in their language
Haptic	Short buzz when it turns green — feel it without looking

4.7 ► The best trick — gate the shutter

Red state → shutter button greyed out and disabled. Green state → shutter turns green and glows.

The artisan doesn't need to understand *anything*. They just move until the button lights up. **Zero literacy required.**

Go further: when green holds stable for 1 second, **auto-capture**. Pressing a button shakes the phone and causes blur — removing the press removes the blur. Small detail, large quality gain.

4.8 Also on-device

- **EXIF/GPS strip before upload** — an artisan's home coordinates must never reach a public listing
- **Offline queue** — village network is intermittent. Shoot 10 products, queue them, auto-upload when signal returns. **Without this the app doesn't work where our users live**
- **Instant rough preview** — show a crude local cutout immediately while the server takes 15s. *Perceived* speed matters more than actual speed

4.9 [!] Capacitor/WebView constraint

In WebView-based frameworks, getting camera frames into JS is slow — you draw video → canvas, then read pixels back, and that readback is the expensive step.

This is exactly why §4.3 is non-negotiable. At 240×180 you get a comfortable 10–15 checks/sec. At full resolution you get ~2 fps and the feature is worthless.

If more speed is needed later, write a small native Android plugin that processes frames on the native side. **For the hackathon, WebView + small buffer is sufficient. Do not get stuck here.**

5. Image processing — server pipeline

5.1 Why server, not phone

Reason	Detail
Phone too small	Target user's phone is ₹7,000, 2–3 GB RAM. A 300 MB segmentation model won't run, or takes 45s and crashes
Two different speed needs	Camera feedback needs <100ms. Final processing can take 20s. Fast→phone, good→server
Model updates	Better model on server = everyone gets it today. On phone = app update, and rural users never update
Battery	Heavy AI on-device drains the phone in 20 minutes. It may be the household's only phone
Equity	Good phone = good photo, cheap phone = bad photo — that violates the entire point of the PS. Server = everyone equal

5.2 Pipeline stages

(1) Quality gate (again, server-side) Reject before spending GPU: resolution < 1000px, blur fail, extreme exposure. Instant feedback, not 30s later.

(2) Segmentation + matting Handicraft problem: **tassels, fringes, jute fibres, jaali work, filigree, loose threads.**

Model	Use
rembg / U ² -Net	Fast, popular. Binary mask — will chop fringes or leave halos
BiRefNet	High-res dichotomous segmentation, much better on fine detail. Baseline choice for textiles
SAM 2	Interactive fallback — artisan taps to indicate the product when auto fails

[!] **Binary mask is not enough.** Semi-transparent edges (muslin, net, chanderi, glass) need **alpha matting** with trimap refinement. Otherwise the pallu appears sliced off.

Benchmark all three on our own test images. Do not trust blog rankings.

(3) White balance / colour correction ← most underrated stage

For textiles this matters more than background removal.

Artisan shoots under a tungsten bulb → whole image orange. In shade → blue. A maroon Sambalpuri saree photographs orange. Buyer orders, receives maroon, returns it, artisan's rating drops.

Options: - Gray-world or Shades-of-Gray (simple, fast) - Learned WB model (better) - ► **Reference card trick** — app says "product ke bagal me ek safed kagaz rakh dein." Calibrate exact white point from that paper, then crop it out. **Zero cost, professional-grade accuracy.**

(4) Exposure & tone Auto-levels, shadow lift, CLAHE for local contrast. **Apply to the product region only** — we already have the mask.

(5) Denoise + selective sharpen Denoise low-light noise, then unsharp mask **on the product only**. Texture (weave, knot, grain) must pop — texture *is* the selling point in handicraft.

(6) Composite on pure white Exact #FFFFFF — RGB(255,255,255). **Verify programmatically:** sample corner pixels and assert. Marketplace systems scan for this and flag near-white values like (252,252,252) even when the eye can't tell.

Shadow policy: **no shadow on main image** (rejection risk). Soft contact shadow allowed on secondary images — looks more real.

(7) Auto-crop to 85% fill Bounding box from mask → compute padding → product occupies ~85-90% → square 2000×2000. **Pure math, no AI** — and it fixes one of the most common rejection reasons.

(8) Perspective correction For flat items — dhurrie, pattachitra, wall hanging, dupatta. Detect quadrilateral → homography warp → flat straight-on view.

(9) Export variants + EXIF strip

Target	Spec
Amazon	2000×2000 square, pure white, JPEG q90, sRGB
GeM	3 images per GeM guidelines [!] <i>specs to be confirmed</i> — \$18
IndiaMART	500×500, max 4MB, product 60–70% of area
Instagram / WhatsApp	1080 square, lifestyle version

5.3 Marketplace rejection triggers — build backwards from these

The pipeline exists to defeat this list:

- Background not pure white (even 252,252,252 gets flagged)
- Product not filling 85% of frame
- Text, watermarks, borders, promotional content on main image
- Blurry or under 1000×1000 (below 1000px, **zoom is disabled — which directly hurts conversion**)
- Inaccurate colour representation vs. what ships
- Shadows or reflections on white background
- Product shown inside packaging
- Over 10 MB, or layers not flattened

Put this mapping in the pitch — each rejection reason → the pipeline stage that prevents it. It shows homework.

5.4 The "Studio" part

PS says "Studio" but never defines it. Our reading — **secondary images**:

- **Lifestyle scene generation** — matka on a wooden table, saree draped on a mannequin. Generative model, **opt-in and capped** (e.g. 2 free per artisan, or unlocked after first sale — otherwise one enthusiastic user burns the budget)
- **Auto detail crops** — weave texture, knot work, hallmark — derived from mask
- **Scale reference** — dimension overlay
- **Wrinkle removal** for textiles

[!] **Generated images are ALWAYS secondary, never the main image.** Marketplaces reject "inaccurate representation," and a generated model shot is exactly that if used as the primary.

5.5 Hard cases specific to handicrafts

Case	Problem
Reflective metal (dhokra, bell metal, brass)	Specular highlights blow out; segmentation confused
Transparent / translucent (glass, muslin, chanderi)	Alpha matting nightmare
Dark product on dark background	Segmentation fails
Fine fringes & tassels	Binary mask chops them
Jewellery	Too small; phone macro struggles

Fallback for all: SAM 2 tap-to-refine, or a "help chahiye" button routing to remote support.

5.6 [!] The line we do not cross

Never change colour or shape. Enhance, don't misrepresent.

Practically: return rate destroys artisan income. Legally: misrepresentation is a listing violation.

→ **Implement "colour lock":** after WB correction, ask the artisan by voice "*kya yeh asli rang hai?*" Publish only on confirmation. Builds trust and keeps us safe.

→ **Enhancement intensity slider.** Handicraft imperfection is a *selling point*. Over-processed photos look factory-made. Let them choose from sterile-studio to natural.

6. Voice & multilingual cataloging

6.1 Requirement recap

Voice note in regional language → translate → SEO-friendly professional description **in English AND Hindi**. Both. Always.

6.2 Flow

```
voice note (regional language)
├ ASR — Bhashini preferred
└ transcript
  └ LLM + image context (vision model)
    { title, desc_en, desc_hi, category, material, dimensions, keywords }
```

6.3 The questions (voice, one at a time)

- *Yeh kya hai?*
- *Kisse bana hai?*
- *Kitna samay laga?*
- *Kya khaas hai isme?*
- *Kitna bada hai?*

6.4 ► Image-first pre-fill — mentor's request, and a friction killer

Mentor asked: "*if image is taken it should tell everything about that image.*"

Implement it. Vision model pre-fills category, material, colour, technique from the photo. The artisan then only **corrects** by voice instead of describing from scratch.

"*Yeh Sambalpuri saree lag rahi hai, cotton ki. Sahi hai?*" → tap yes → done.

Massive reduction in effort, and it directly serves PS feature 2.

6.5 Structured manufacture information

Mentor's note #3. Capture as structured fields, not free text:

```
material · technique · dye type · loom type · time taken · cluster · GI claim
```

Feeds three things at once: pricing (raw material costs — PS explicitly requires this), the craft story (\$13.5), and authenticity (\$13.6). **High value, cheap to build.**

6.6 Language stack

Bhashini (MeitY) — government language-AI mission covering ASR, translation, TTS for Indian languages.

Why preferred over commercial APIs: government-aligned stack, better Indian dialect coverage, dramatically lower cost.

[!] **UNVERIFIED** — access terms, rate limits, whether our use case qualifies. See §18.

6.7 Voice is not optional anywhere

Every screen that asks anything must speak it. Every error must be spoken. Every confirmation must be spoken. **Text is the fallback, not the default.** This is what "accessible layouts" means for a low-literacy user.

7. Dynamic pricing

7.1 The problem

No training data exists for "what should this handicraft cost." A pure ML price predictor with no training set is a slide we cannot defend.

7.2 ► Solution — cost-up, not market-down

Start from what it cost to make, then show the market range.

This matches the PS wording exactly — "raw material costs" is named as an input.

Four layers:

(1) Raw material cost (*PS requirement, and easiest*) Yarn, silk, zari, clay, brass rates. Sources: handloom raw material supply scheme rates, NHDC yarn prices, cluster-level input costs. **Or simply ask by voice** — "dhaaga kitne ka aaya?"

(2) Labour hours × cluster wage rate. Artisan states hours by voice.

(3) Market comparables Query similar listings by category + material + size. Amazon and Flipkart catalog/search APIs (we're already authenticated where connected). GeM publishes rate contracts.

(4) [CRITICAL] Floor-price guard — the most important part

If cost + labour > suggested market price → **warn them they are about to sell at a loss.**

Under-pricing is the actual epidemic in this sector, not over-pricing. This single feature does more for the "increase annual income" impact goal than any other.

7.3 [!] GeM discount interaction — do not miss this

GeM **mandates a minimum discount off MRP** when listing (~10% typical).

Our floor-price guard must run AFTER that discount, not before.

If we suggest ₹2,000 and GeM knocks 10% off, the artisan nets ₹1,800 — which may be below cost. **Set MRP such that the post-discount price still clears the floor.** Miss this and we recommend loss-making prices on the platform we're headlining.

8. Channel integration layer

8.1 Reality check — the tiering nobody tells you

Not all platforms are equally connectable.

Platform	Public 3rd-party API?	Buildable?	Verdict
GeM	No API — Excel bulk upload	Yes	Tier 1 — PRIMARY
ONDC	Open protocol, you become a node	Yes	Tier 1 — core
Amazon	SP-API, documented, sandbox	Yes	Tier 1 — demo
Flipkart	Seller API, OAuth, sandbox	Yes	Tier 1 — demo
Meesho	Partner-only	No	Tier 2 — assisted
IndiaMART	🔑 Lead-focused, not listing	Partial	Tier 2
Myntra	Brand-gated, invite-only	No	Tier 3 — DROP
Nykaa / Ajio	Curated, mostly inventory model	No	Tier 3 — DROP

[!] Drop Myntra and Nykaa from all slides. Myntra onboarding requires GST, PAN, Aadhaar, **trademark certificate or brand authorisation letter**, address proof and incorporation documents. A weaver has no trademark. If a judge from industry is present, claiming Myntra integration is instant credibility loss.

8.2 Amazon — SP-API

Prerequisite: artisan must already have a Seller Central account. **SP-API cannot create one.**

Setup: register as SP-API developer → build a **public application**. Public apps must be listed in the Selling Partner Appstore per the developer agreement.

Auth (Login with Amazon — Amazon's OAuth 2.0):

1. Artisan taps "Amazon se jodein"
2. Redirect to Amazon consent page (signs in if needed)
3. Consent → Amazon returns spapi_oauth_code to our redirect_uri
4. Exchange code at LWA token endpoint → refresh token
5. Store encrypted; mint short-lived access tokens per call

APIs used: | API | Purpose | |---| | Product Type Definitions | **Call first** — gets the category schema (what attributes a saree needs) | | Listings Items | PUT /listings/2021-08-01/items/{sellerId}/{sku} | | Feeds | Bulk operations | | Orders | Order retrieval | | Notifications | Push → our queue (\$9) | | Reports | Inventory reconciliation |

Sandbox exists — this is our hackathon lifeline. Full flow demoable without a real seller account.

[!] **Public app publication requires Amazon review** (data protection policy, security assessment). **Not achievable in 36 hours.** Demo on sandbox and state honestly: "*sandbox-validated, production pending Appstore review.*"

8.3 Flipkart — Marketplace Seller API

Cleaner third-party documentation than Amazon.

Two app types: `self_access_application` (own resources) and `third_party_application` (accessing other sellers' resources on their behalf). **We need third-party.**

Registration: register on the Partner Dashboard, answer **Yes** to the "API Partner" question. Flipkart's team contacts you to verify within ~72 hours.

Auth (Authorization Code):

```
https://api.flipkart.net/oauth-service/oauth/authorize
?client_id=<id>&redirect_uri=<uri>
&response_type=code&scope=Seller_Api&state=1234

→ seller logs in at seller.flipkart.com, clicks Allow
→ authcode returned

curl -u <appid>:<app-secret> \
https://api.flipkart.net/oauth-service/oauth/token
?redirect_uri=<uri>&grant_type=authorization_code
&state=<state>&code=<code>

→ { access_token, token_type: bearer, refresh_token,
    expires_in: 5183999, scope: Seller_Api }
```

[CRITICAL] **Critical gotcha:** `expires_in: 5183999 seconds` $\approx$ **60 days**. Never hard-code the token. **Build an automated refresh job.** Teams forget this and everything silently breaks two months later with 401s.

Sandbox available and testing there is a prerequisite before production. Official Java SDK on GitHub.

8.4 ► GeM — PRIMARY channel

No public seller API. The Excel bulk upload IS the integration — and it's an official documented path, not a workaround.

Per GeM's Catalogue Management guidance: sellers may use the bulk upload facility with the **category-specific Excel format**, and once filled accurately and saved, **the item is automatically listed on GeM 3.0**, subject to accuracy and completeness.

So our job: generate a perfect, category-correct GeM Excel from a photo and a voice note.

Why this is our strongest AI story

GeM has over 10,700 product categories. Choosing the wrong category is the single most common listing failure. Errors in category selection and attribute accuracy cause delayed approvals, repeated rejections, and lost opportunities.

An artisan cannot navigate 10,700 categories. Neither can most educated sellers — an entire consultancy industry exists purely to fill GeM catalogues correctly.

Our vision model maps a photo of a Sambalpuri saree to the right GeM category. Our NLP fills the attribute schema. We look up the HSN code.

That is an entire consultancy industry, automated, free, in the artisan's language. Background removal is a commodity. **This is not.** This is what "AI-driven market linkage" actually means.

Fields the adapter must fill

GeM field	Source
Category / sub-category	Vision model → GeM taxonomy
Product name & description	NLP from voice note
Brand	Artisan's own name / cluster name (or request new brand)
Material, physical parameters	Voice note + structured manufacture fields
Certifications	Handloom Mark / GI / Craft Mark if held
HSN code	Auto-lookup from category
SKU ID	We generate
Country of origin, local content	India, 100% (required for OEM)
MRP & offer price	Pricing engine ([!] see §7.3)
Images	3, per GeM specs

GeM facts — do not get these wrong

Fact	Detail
Seller API	None. Category Excel bulk upload; auto-publishes if correct
Registration	Free. Aadhaar or PAN verification. Entirely online, paperless, 1-3 working days
Caution money	Artisans & weavers FULLY EXEMPT. Also MSE women, MSE SC/ST, SHGs, DPIIT startups
GST	Generally required, but artisans are an exempt category. Proprietors can start with PAN and add GST later
Transaction charges	Zero up to ₹10 lakh order value. 0.30% from ₹10L-₹10cr. Flat ₹3L above ₹10cr
Annual Milestone Charge	One-time ₹10,000 + tax once Seller Merchandise Value crosses ₹20 lakh in a FY; charges only apply past that
Mandatory discount	Minimum discount off MRP required when listing (~10% typical). Slab discounts on bulk optional
Seller type	Artisan = OEM (makes own goods), not Reseller
Top rejection cause	Name mismatch across Aadhaar / PAN / GST / bank. Pre-check spelling consistency before they start
MSE quota	Minimum 25% of government procurement reserved for MSMEs — guaranteed demand
Scale already	~ 1.5 lakh weavers and handloom entities already on GeM

[BONUS] **Pitch consequence:** an artisan will never cross ₹20 lakh SMV. **GeM is completely free for our users** — no caution money, no transaction charges, no milestone charge, free registration. The only remaining barrier is *knowing how*. **That is exactly the barrier we remove.**

[!] **UNVERIFIED — must confirm:** Vendor Assessment fee (~₹11,200 + GST) applies to manufacturers/OEMs, and OEM registration is described as mandatory for listing. **Artisans are OEMs. Are they exempt?** Get it in writing from GeM support. If not exempt, it's a real barrier needing a cluster-sponsored path.

8.5 ONDC — network node

We don't consume an API. **We become a node.**

Onboarding: generate self-signed certificates with **separate key pairs for signing and encryption**; separate certs for buyer node (BAP) and seller node (BPP). Register on the registry portal with Subscriber ID, Country, Cities, Domain, Type. Subscription workflow runs; ONDC registrar approves. You then hold `subscriber_id`, `signing_public_key`, `encryption_public_key` — used in every request.

Our role: Marketplace Seller Node (MSN) — an MSN doesn't hold its own inventory, it offers other sellers' goods. Exact fit.

Why it's powerful: a seller onboarded once through any Seller Network Participant becomes **automatically visible across all Buyer Network Participants**. One integration, many buyer apps. Amazon offers nothing comparable.

Staging environment exists — most demoable of all channels.

8.6 Meesho — assisted only

No open third-party API; integration is partner-gated.

But onboarding is the friendliest for our users: supplier.meesho.com or Supplier Hub app → mobile OTP → three blocks (business details incl. GSTIN if any; identity + banking with PAN, account, IFSC; pickup address). **Meesho supports sellers without GSTIN in several GST-exempt categories** under government provisions for small online sellers. Sellers without a registered brand can list under **"Unbranded."**

Our approach: generate the copy-paste block, deep-link to Supplier Hub, voice-guide step by step. Manual — but still far easier than unaided.

8.7 GST — the wall, and the way through

[KEY] **Most artisans do not need GST registration at all.**

Under **Notification 34/2023-Central Tax (effective 1 Oct 2023)**, persons supplying goods through e-commerce operators are **exempt from mandatory GST registration** if:

- (a) supplies are within a single State/UT
- (b) no inter-state supply
- (c) they have a PAN
- (d) PAN is declared on the GST portal with place of business and State/UT
- (e) they obtain an **enrolment number** on the common portal after PAN validation — and cannot supply through an ECO before that

Exemption applies where aggregate turnover in the previous and current FY is below the State/UT registration threshold. GSTN has built the enrolment functionality.

→ **Build the GST Enrolment Wizard.** Voice questions: "Kis rajya me hain? Doosre rajya me bhejenge? Saal me kitna kaam?" → route:

Situation	Path
Intra-state only, below threshold	No GST needed — enrolment number only. Wizard walks them through
Inter-state intent	Full GST registration required — no exemption
Above threshold	Full registration

This feature alone can win the round. It literally, legally addresses "drastically lower the barrier to entry" — and almost no other team will know this notification exists.

8.8 Adapter architecture

```
Product (canonical)
  ↓
Channel Adapter Layer ← plugin pattern, one per channel
├─ GeMAdapter          → category .xlsx
├─ AmazonAdapter       → field list / SP-API payload
├─ FlipkartAdapter     → Seller API payload
├─ ONDCAdapter         → Beckn catalog
├─ MeeshoAdapter       → copy-paste block
├─ WhatsAppAdapter     → image + caption
└─ MarketplaceAdapter → direct DB write
```

Each adapter: `map_category()` · `map_attributes()` · `format_images()` · `render()` . Adding a channel = adding a plugin, never touching core.

9. Order management — unified inbox

9.1 Why this is non-negotiable

Without it, "virtual business manager" is false advertising. If an artisan must log into Seller Central to see orders, we have failed the PS.

9.2 Ingestion — two modes, one canonical object

Channel	Mechanism	Reality
Our marketplace	Direct DB write	Instant — we own it
ONDC	Beckn callbacks (<code>on_confirm</code> , <code>on_status</code>)	True push — best case
Amazon	Notifications API → our queue; Orders API as fallback poll	Near-real-time
Flipkart	Orders API polling [!] <i>verify if webhooks exist</i>	Scheduled
GeM	No API	See below
Meesho	No public API	Manual

Architecture:

```
webhook receivers ┌
                  │ → normalize → canonical Order
scheduled pollers └
                  ↓
                state machine: placed → packed → shipped
                              → delivered → settled
                              ↓
                artisan inbox (their language + voice notification)
```

9.3 [!] The GeM gap — handle honestly

GeM orders appear on the GeM seller dashboard and nowhere else. Options, in preference order:

1. **Cluster coordinator / remote support** checks the GeM dashboard and marks orders in the admin console (§3.2). A real, fundable job — **remote, not a field visit**
2. **Email parsing** — if GeM sends order notification emails, parse them
3. **Roadmap**: "GeM order API pending MoU"

Do not pretend we have GeM order sync. Stating the gap and showing the workaround reads as maturity. Faking it is a question we cannot survive.

10. Inventory sync

(PS requirement: "seamlessly digitize their inventory")

10.1 The real problem

Not "decrement everywhere." It's **race conditions on unique items**.

Two buyers, two platforms, the same one-of-a-kind saree, the same second. Propagation to Amazon and ONDC takes seconds to minutes. **That window cannot be closed.** Every multichannel tool in the world lives with it.

10.2 What we build

- **Central inventory ledger** — single source of truth in our DB. **No channel is authoritative**
- **Atomic reserve-on-order** with optimistic locking (version column). First write wins; second rejected
- **Immediate fan-out** to all channels on any change
- **Oversell resolution flow** — auto-cancel the losing order, apology + priority-restock offer to that buyer, and **never let the artisan absorb the account-health hit silently**

10.3 ► The mitigation that actually shrinks the problem

Most handicrafts are replicable, not unique. A weaver can make another gamcha; a specific antique brass piece he cannot.

Split the model:

Type	Behaviour
Made-to-order (qty > 1 + lead time)	No race condition at all
Unique piece (qty = 1)	Reserve-and-race, small window

Push artisans toward made-to-order wherever the craft allows. Better for them too — they never turn down demand.

11. Settlement visibility

11.1 Sources

Channel	Source
Amazon	Finances API (financial events) + settlement reports
Flipkart	Settlement/payments endpoints [!] <i>verify</i>
ONDC	Per network agreement; RSP handles reconciliation
Our marketplace	Payment gateway settlement webhooks
GeM	No API — government pays seller directly

11.2 [!] The honest limit

We can see marketplace settlement **schedules and promises**. **We cannot see the artisan's bank account.**

So the UI language matters enormously:

"₹4,200 aa gaye." **"Amazon ne ₹4,200 bheje hain — 15 tareekh tak aane chahiye."**

11.3 ► The confirm tap

Add: **"Paisa aaya?"** → yes/no.

One tap. That gives us real settlement-delay data across thousands of artisans — **genuinely valuable evidence to hand back to the ministry.**

Turn the limitation into our best dataset.

12. Logistics & packaging

Goal: get the outcome without becoming a logistics company.

12.1 Three fulfilment models, chosen per order

Model A — Marketplace-managed (*default wherever available*) Amazon Easy Ship / Flipkart FBF. Courier collects from artisan, marketplace generates label and owns the SLA. **Zero logistics work for us.**

Model B — Aggregator-managed (*our marketplace + ONDC orders*) One shipping aggregator (Shiprocket / Delhivery / DTDC). Generate AWB, book pickup, track. **One integration, not an operation.**

Model C — ► Cluster Hub (*this is the one that solves packaging*)

Artisans bring goods to a Common Facility Centre. **These already exist** as funded government infrastructure (Block Level Clusters, Handicraft Service Centres).

Hub does: **quality check → professional packing → weighing/measuring → single consolidated courier pickup.**

Look what this solves simultaneously: - Packaging quality — one trained person, proper materials, vs 50 artisans guessing - **Quality check** — answers mentor Q7 (\$13.7). A human inspects - Pickup economics — one courier stop for 50 artisans, not 50 rural pickups - Weight/dimension accuracy — prevents silent margin loss - Creates a local job - **Uses infrastructure the government already paid for**

Don't build a logistics company. Plug into what exists.

12.2 In-app packaging support

- **Per-craft packing SOP** — short voice + video per product type. Textiles (moisture, tissue, poly), terracotta (bubble wrap, corner protection, fragile marking), metal (anti-tarnish paper), wood (humidity). **Generate once per category, reuse forever**
- **Packing kit checklist** — materials to keep on hand
- **Auto-generated shipping label** with QR → Digital Product Passport (\$13.6). Ties provenance to something physical
- **Premium packaging tier** — branded artisan tags, craft story card in the box

12.3 [FIX] Loose ends — do not leave these open

Loose end	Handling
Pin code serviceability	[!] Check BEFORE the artisan lists. If their pincode isn't serviceable, block self-ship channels and route to Cluster Hub only. Discovering this after a sale is a disaster
Weight/dimension errors	Courier surcharges for under-declared weight silently eat margin. Hub weighing solves it. For self-ship, force weight entry with category defaults
COD	Huge in India, huge RTO risk. Prepaid-only at launch. Artisans cannot absorb return-to-origin costs
RTO (return to origin)	The real killer of Indian e-commerce economics. Track RTO rate per artisan/category. Address-quality checks upfront
Returns & reverse logistics	Who inspects the returned item? → Cluster Hub. Who bears cost? → must be defined per channel and shown to the artisan BEFORE they list
Transit damage	Shipping insurance for fragile categories. Terracotta and glass need it
Who pays shipping	Buyer pays forward. Return-shipping liability defined explicitly per channel — never a surprise
Perishable/fragile flag	Set at catalog time; drives packing SOP, insurance, and courier choice automatically

13. Mentor notes — answered

13.1 "How do we ensure their product doesn't have a cheap copy?"

Split it — two different problems:

(a) **Someone copies the artisan's design** → IP protection. Very hard, arguably out of scope. **Say so honestly.**

(b) **Fake products sold as authentic on our platform** → solvable, and **the government already solved it.**

Existing certification marks: **Handloom Mark, India Handmade Mark, Craft Mark, Silk Mark, and GI tags.** The **Handlooms (Reservation of Articles for Production) Act, 1985** reserves 11 specified textile articles for handloom production.

Do not invent a new authenticity system. For a government PS, "we surface the Handloom Mark and GI status that DC-Handlooms already issues" beats any novel scheme.

13.2 "Recommendation system"

Two different systems — separate them: - **Buyer-side** ("goes well with this") → marketplace feature, lower priority - **Seller-side** ("artisans like you sell more when they add X") → supports "virtual business manager". **Build this one**

13.3 "If image is taken it should tell everything about that image"

Doing it. See §6.4 — vision model pre-fills, artisan corrects by voice. Friction killer.

13.4 "Complete manufacture information"

Doing it. See §6.5 — structured fields feeding pricing, story, and authenticity.

13.5 "How are we going to show the story"

Decision: artisan tells it by voice, AI structures it. **We do not build a content team.** Story lives on the marketplace product page (§3.3) and the packaging story card (§12.2).

13.6 "Product validated or not?"

Answer = government certification marks (§13.1) + **Digital Product Passport** (§13.8).

13.7 "Who does the quality check?"

Be honest: we cannot. A software team cannot do physical QC at scale.

The real answer, three layers: 1. **Cluster / Common Facility Centre does physical QC** — they already do it for exhibitions, and it's the natural Cluster Hub function (§12.1 Model C) 2. **Platform-level** — buyer ratings, returns tracking, delisting on repeat complaints 3.

Certification marks where held

Inventing a fake answer here is worse than admitting the limit.

13.8 "Include blockchain to make the business decentralized"

[CRITICAL] Say this to the mentor carefully:

Blockchain cannot verify that a physical saree is handloom. It's a tamper-evident ledger. If someone writes "authentic Sambalpuri" about a powerloom fake, the chain faithfully stores that lie forever. **This is the oracle problem and it is unsolvable at the chain layer.**

But there is a narrow, honest version worth building:

Digital Product Passport — a signed record containing artisan/Pahchan ID, cluster, craft type, materials, GI claim, date, and image hashes. **Signed by the cluster office or handicraft board** (the trusted authority). Hash anchored to a public chain or transparency log so it cannot be backdated or altered. **QR code on the physical tag links to it.**

Trust comes from WHO SIGNED, not from the chain. The chain only proves nobody edited it afterward.

Pitch as: "Blockchain-anchored provenance, with certification authority remaining with DC-Handicrafts." Satisfies the mentor, technically honest, and answers §13.1 and §13.6 in one stroke.

[!] **Do NOT make it core. Do NOT put it in the main architecture diagram. One slide, clearly scoped.**

13.9 "Samaan package kaise hoga seller ke end se?"

Answered in full — §12.

13.10 "Who other people have worn/used those products"

Social proof / UGC. **Marketplace feature, not artisan app. Parked** — revisit only if time allows.

14. Privacy & DPDP

14.1 [KEY] Golden rule

What we don't store cannot leak.

14.2 What we NEVER store

PAN number · Aadhaar number · GST certificate · bank account number · photos of any document · platform passwords · platform OAuth tokens (exception: Amazon/Flipkart refresh tokens if the artisan explicitly connects — encrypted)

14.3 What we DO store

Phone + OTP session · language preference · display name · UPI ID (payout only) · **boolean readiness flags** · product catalog · self-reported outcomes

The boolean design is the key decision. `has_pan: true` — never the number. This keeps us out of DPDP's heaviest obligations entirely.

14.4 Design principles

1. **Never ask for credentials.** OAuth redirect only. If any team demo shows "enter your Amazon password," that's a disqualification-level flaw
2. **Verify, don't store.** If PAN/GST verification is ever needed, use a regulated KYC provider and store `verified: true` + timestamp + masked last-4. Never the full number
3. **DigiLocker** where verified documents are genuinely required — consent-based government fetch, no document photos [!] *requester-org registration is a real gate* — §18
4. **[NEVER] Do not touch Aadhaar.** Aadhaar Act restricts storage. If ever unavoidable, Offline eKYC XML or Virtual ID only. **Simplest advice: avoid entirely**
5. **Bank verification via penny-drop** (₹1 test transfer), token stored, not the account number

14.5 DPDP status and obligations

DPDP **Rules notified 14 November 2025**. Phased: - **November 2026** — Rule 4 (Consent Management) — **first hard deadline - May 2027** — Rules 3, 5-16, 22, 23: notices, security safeguards, breach notification, data erasure, children's data, rights management, cross-border transfers

Penalties: Data Protection Board can impose up to **₹250 crore** for serious violations.

Breach: two-stage — notify the Board **immediately**, then affected Data Principals **within 72 hours** with plain-language description of what was exposed and protective measures. Failure to notify: up to **₹200 crore**.

14.6 ► The multilingual consent tie-in

DPDP requires consent notice in **plain language**, with the user able to choose from the 22 Eighth Schedule languages.

Our app is already multilingual. So: play the consent notice **by voice, in Odia / Hindi / Bengali** — "*hum aapka PAN sirf GST enrolment ke liye istemaal karenge, kisi aur ke saath share nahi karenge.*"

Store a consent artifact: timestamp + language + notice version.

One feature, two wins: DPDP compliance AND accessibility. Put this on a slide — almost no team will think of it.

15. Data model

```

Product {
  id
  artisan_id
  images[]      { url, size_variant, is_primary }
  title
  desc_en
  desc_hi
  category      // internal taxonomy
  category_map { gem_id, amazon_node, ondc_code, meesho_cat }
  material
  technique
  dye_type
  loom_type
  time_taken_hours
  dimensions
  weight
  is_fragile
  hsn_code
  cost_material // artisan-stated
  labour_hours  // artisan-stated
  floor_price   // computed
  mrp
  price
  quantity
  is_made_to_order // true = no race condition
  lead_time_days
  gi_claim
  certifications[] // handloom_mark, craft_mark, silk_mark, gi
  created_at
}

Artisan {
  id
  phone
  language
  display_name
  cluster_id
  pincode      // for serviceability check
  upi_id       // payout only
  // readiness FLAGS ONLY - never the values
  has_pan      : bool
  has_bank     : bool
  has_gst      : bool
  has_artisan_card : bool
  intra_state_only : bool
}

ChannelStatus {
  artisan_id
  channel      // gem | amazon | flipkart | meesho | ondc | whatsapp
  signup_status // not_started | taught | self_reported_done
  products_exported[]
  last_export_at
}

Order {
  id
  channel
  external_order_id
  artisan_id
  product_id
  quantity
  amount
  state      // placed | packed | shipped | delivered | settled
  fulfilment_model // A_marketplace | B_aggregator | C_cluster_hub
  expected_settlement_date
  artisan_confirmed_payment : bool // the "paisa aaya?" tap
  created_at
}

InventoryLedger {
  product_id
  available_qty
  reserved_qty
  version      // optimistic locking
  updated_at
}

```

16. Tech stack

Layer	Choice	Notes
Mobile	React + Vite + Capacitor	[!] WebView frame constraint — \$4.9
Admin web	React	Multi-tenant
Marketplace	React (SSR preferred for SEO)	Public catalog pages need to be indexable
Backend	<i>(team to confirm)</i>	Must support webhook receivers + scheduled pollers
Segmentation	BiRefNet baseline, SAM 2 fallback	Self-hosted — \$16.1
ASR / translate / TTS	Bhashini [!] <i>unverified</i>	Fallback: commercial API
LLM	<i>(team to decide)</i>	Description generation + category mapping
Generative images	API, opt-in, capped	Secondary images only
Queue	Required for Amazon Notifications	
Payments	Razorpay / Cashfree	Settlement webhooks + penny-drop
Shipping	One aggregator	Shiprocket / Delhivery / DTDC

16.1 Image approach — recommendation

Layered, and the layer that matters is the boring one.

- **Background removal + colour + crop → self-hosted model.** This runs on every photo. Per-image API pricing kills unit economics, and we can't show cost-per-artisan if we don't control it
- **Generative "saree on a model" → API, opt-in, hard-capped.** Genuinely a gamechanger for demo, but **secondary image only** — main image must never be generated (misrepresentation = rejection)

The generative feature is a demo weapon, not a product foundation. 20 seconds on stage. Build the boring pipeline properly.

17. Open decisions register

#	Decision	Options	Owner	Status
1	Image approach	Self-hosted / API / hybrid		🕒 Recommendation in §16.1
2	Backend framework			● Open
3	LLM choice			● Open
4	Framing check method	Grid variance vs TFLite model		🕒 Recommend grid variance
5	Stage-3 trigger point	After N listings / first sale / manual		● Team must argue this — too early = wall moved, too late = never unlocks
6	Stage-2 payout floor	UPI-only, or cash/cluster path?		● Open
7	Marketplace SSR vs SPA			🕒 SSR recommended
8	Which languages at launch			● Open
9	Blockchain scope	One slide only vs implemented DPP		🕒 Recommend one slide

18. Verification checklist

[NEVER] **Nothing here goes on a judged slide until it's ticked.**

#	To verify	Where	Owner
1	● Download 2-3 real GeM category Excel templates (handloom/handicraft) — the entire render layer builds against these schemas	GeM Catalogue Management	
2	● GeM handicraft/handloom category tree — what the vision model maps into	GeM	
3	● Vendor Assessment (~₹11,200) — are artisans/weavers exempt? Get it in writing	GeM support	
4	🕒 GeM exact image specifications	GeM	
5	🕒 Bhashini access terms, rate limits, use-case eligibility	bhashini.gov.in	
6	🕒 DigiLocker requester-organisation registration requirements	DigiLocker	
7	🕒 Flipkart — do webhooks exist, or polling only?	Flipkart API docs	
8	🕒 Flipkart settlement/payments endpoints	Flipkart API docs	
9	🕒 GeM caution-money & GST exemption scope for artisans — confirm from primary source	gem.gov.in	
10	🕒 PM Vishwakarma — beneficiary count, trade taxonomy, data access	pmvishwakarma.gov.in	
11	🕒 Confirm current handloom/handicraft artisan population figures	PIB / ministry annual reports	

19. Reference numbers

[!] **All figures below need primary-source confirmation before use on a slide (§18 #11).**

Figure	Value
Handloom weavers & allied workers	~ 35.22 lakh
Handicraft artisans (census)	~ 68.86 lakh
Women handicraft artisans registered with DC-Handicrafts	~ 16.87 lakh
Weavers/handloom entities already on GeM	~ 1.5 lakh
Handloom exports 2025-26	~ ₹1,330.96 crore
GeM product categories	10,700+
CSC Village Level Entrepreneurs	~ 5.4 lakh (≈4.35 lakh in gram panchayats)

► The IndiaHandmade number — our thesis in one statistic

IndiaHandmade.com already exists. Government-run, direct-to-consumer, for verified weavers and GI-tagged products.

It onboarded **4,186 weavers and artisans between 2023-24 and June 2026.**

Against ~35.22 lakh handloom workers, that is **≈0.1% penetration in three years.**

This is not a threat — it's our entire thesis, handed to us. The government already built the storefront. It didn't scale because **the storefront was never the bottleneck. The bottleneck is getting an artisan's product READY to be listed.**

Pitch line:

"IndiaHandmade proves the demand side works. In three years it onboarded 4,186 artisans out of 35 lakh. We're not building another storefront — we're building the onboarding layer that fills every storefront, including that one."

Appendix — Sources

GeM: gem.gov.in Catalogue Management; GeM Revenue Policy (eff. 9 Aug 2024) via PIB/NewsOnAir/IBEF; GeM registration guidance (multiple secondary — needs primary confirmation) · Amazon: SP-API developer documentation, LWA authorisation workflow · Flipkart: Marketplace Seller API documentation, Partner Dashboard · ONDC: network participant onboarding documentation · Meesho: Supplier Hub onboarding guidance · GST: Notification 34/2023-Central Tax · DPDP: MeitY Rules notified 14 Nov 2025 · Handloom/handicraft figures: PIB, Fourth All India Handloom Census 2019-20 · IndiaHandmade: National Handloom Day 2026 coverage · CSC: csc.gov.in · Certification marks: DC-Handlooms, Handlooms (Reservation of Articles for Production) Act 1985

[!] Several figures come from secondary sources. **Confirm against primary government sources before any judged presentation.**